Guía de Práctica – Semana 5

ESP32 en Wokwi: 2 LEDs y 2 botones

Mtro. Zeus Cobian · Internet de las Cosas (V0748)

Objetivo: Familiarizarse con el uso de ESP32 en Wokwi y la lectura de entradas digitales para controlar salidas (LEDs) mediante botones.

Requisitos previos: Cuenta en Wokwi y nociones básicas de Arduino. Usaremos el entorno Arduino en Wokwi.

1) Crear el proyecto

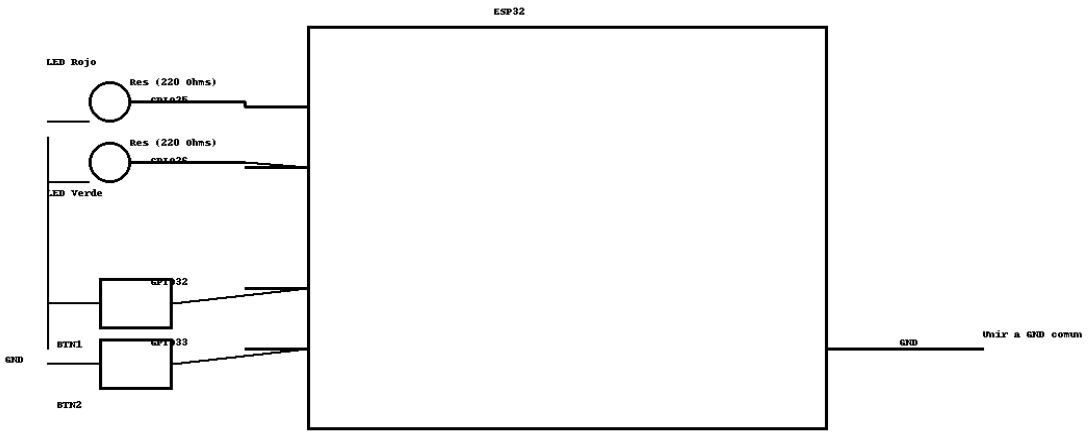
Ingresa a https://wokwi.com → New Project → busca 'ESP32' y selecciona 'ESP32 Arduino'. Guarda el proyecto con el nombre 'ESP32-LEDs-Botones'.

2) Añadir componentes

Agrega desde el panel de piezas: 2 LEDs (rojo y verde) y 2 push buttons. Para esta práctica usaremos las resistencias internas de pull-up del ESP32, por lo que no añadiremos resistencias externas para los botones.

3) Cableado (Wokwi)

• LED rojo al pin GPIO25 (ánodo al pin, cátodo a GND) con resistencia de 220 Ohms.
• LED verde al pin GPIO26 con resistencia de 220 Ohms.
• Botón 1 entre GPIO32 y GND (usaremos INPUT_PULLUP).
• Botón 2 entre GPIO33 y GND (usaremos INPUT_PULLUP).
• Conecta GNDs comunes. Consulta el diagrama de cableado.



4) Cargar el código

Copia el siguiente código en main.ino (Arduino) dentro de Wokwi y presiona 'Play' para simular.

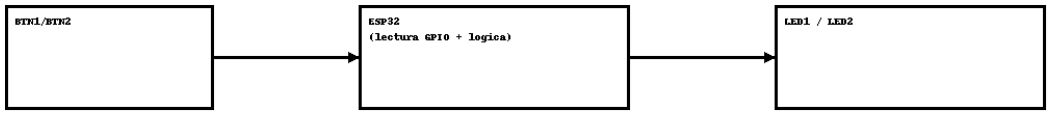
Código Arduino (Wokwi)

```
#define LED1 25
#define LED2 26
#define BTN1 32
#define BTN2 33

void setup() {
```

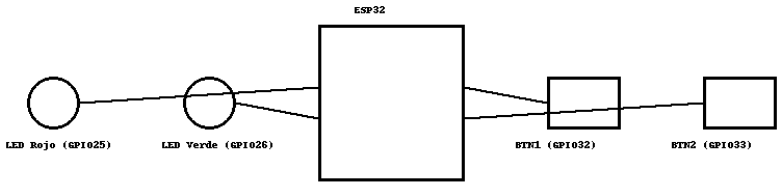
```
  pinMode(LED1, OUTPUT);
  pinMode(LED2, OUTPUT);
  pinMode(BTN1, INPUT_PULLUP); // activo en LOW
  pinMode(BTN2, INPUT_PULLUP); // activo en LOW
}

void loop() {
  if (digitalRead(BTN1) == LOW) {
    digitalWrite(LED1, HIGH);
  } else {
    digitalWrite(LED1, LOW);
  }

  if (digitalRead(BTN2) == LOW) {
    digitalWrite(LED2, HIGH);
  } else {
    digitalWrite(LED2, LOW);
  }
}
```

Cómo funciona

Usamos las resistencias internas de pull-up del ESP32 en los pines 32 y 33. Cuando el botón NO está presionado, la entrada lee HIGH; al presionarlo, el pin se conecta a GND y lee LOW. Con esa lógica encendemos cada LED.



Vista conceptual del montaje



Pruebas sugeridas

• Presiona BTN1 y verifica que enciende solo el LED rojo. Suelta y debe apagarse.
• Presiona BTN2 y verifica que enciende solo el LED verde.
• Mantén presionados ambos botones: deben encender ambos LEDs.
• Cambia el código para que cada botón *altere* el estado del LED (toggle) en lugar de mantenerlo presionado.

Solución (toggle básico)

```
bool led1State = false;
bool led2State = false;

void loop() {
  static bool last1 = HIGH, last2 = HIGH;
  bool b1 = digitalRead(BTN1);
  bool b2 = digitalRead(BTN2);

  if (last1 == HIGH && b1 == LOW) { // flanco de bajada
    led1State = !led1State;
    digitalWrite(LED1, led1State ? HIGH : LOW);
  }
  if (last2 == HIGH && b2 == LOW) {
    led2State = !led2State;
    digitalWrite(LED2, led2State ? HIGH : LOW);
  }
  last1 = b1;
  last2 = b2;
  delay(30); // anti-rebote simple
}
```

Notas para el docente

• Este ejercicio está listo para integrarse con MQTT en la Clase 2 (publicar estados en un topic y suscribirse a comandos para encender LEDs). • Si Wokwi marca errores de pines, cambia GPIO25/26 por GPIO2/4, según el modelo de placa virtual. • Recuerda que INPUT_PULLUP invierte la lógica (presionar = LOW).